

3D LIBRARIES ANALYSIS

1. METHODOLOGY

Here is the methodology used to compare and choose the best 3D library. As we already chose to use Python as our programming language, we had to look for 3D libraries based on python. After doing some research, I chose to compare 3 libraries:

- modernGL
- panda3D
- ursina

But then, I had to find a way to test them by doing similar experiments for a more accurate and balanced conclusion. I then asked ChatGPT to generate one code for every library doing the same thing. The generated code was product with additional print statements to compare rational and objective results.

2. FIRST OBSERVATIONS

The first things I noticed were the codes; their difficulty and size were different. Only by these observations, I was able to start with a small ordering of the libraries that looked easier to use:

1. ursina
2. panda3D
3. modernGL

Then, I downloaded all the libraries and tried to run them. For some of them, I had some errors. Here is a classification of libraries from no errors to main errors:

1. panda3D
2. modernGL
3. ursina

For course, these errors are not the most meaningful characteristic to take in account, but it could still be a nice way to know what we could expect from these libraries for errors.

3. EXPERIMENTS

The codes are used to display a moving set of cubes. In each code, we can change the number of cubes to test the limits of the library. Here are the kind of output I got from these experiments:

```
=====
URSINA RESULTS
=====
Number of cubes: 100
Frames measured: 1799
Average FPS: 59.93
Average frame time: 16.685 ms
Median frame time: 16.662 ms
P95 frame time: 18.417 ms
P99 frame time: 20.542 ms
=====
```

```
=====
PANDA3D RESULTS
=====
Number of cubes: 100
Frames measured: 45974
Average FPS: 1532.36
Average frame time: 0.653 ms
Median frame time: 0.532 ms
P95 frame time: 1.129 ms
P99 frame time: 2.552 ms
=====
```

```
=====
MODERNGL RESULTS
=====
Number of cubes: 100
Frames measured: 3598
Average FPS: 119.88
Average frame time: 8.341 ms
Median frame time: 8.333 ms
P95 frame time: 13.267 ms
P99 frame time: 14.088 ms
=====
```

At first, I didn't really go through all these values. I wanted to know what they looked like. With only few cubes, I thought panda3D and modernGL were as good but ursina was a bit behind. When I was running ursina, the cubes were not moving, and I had to type "control C" in the terminal to see it move which is not very practical. When I increased the number of cubes to 1000 and beyond, the differences where even more obvious. With

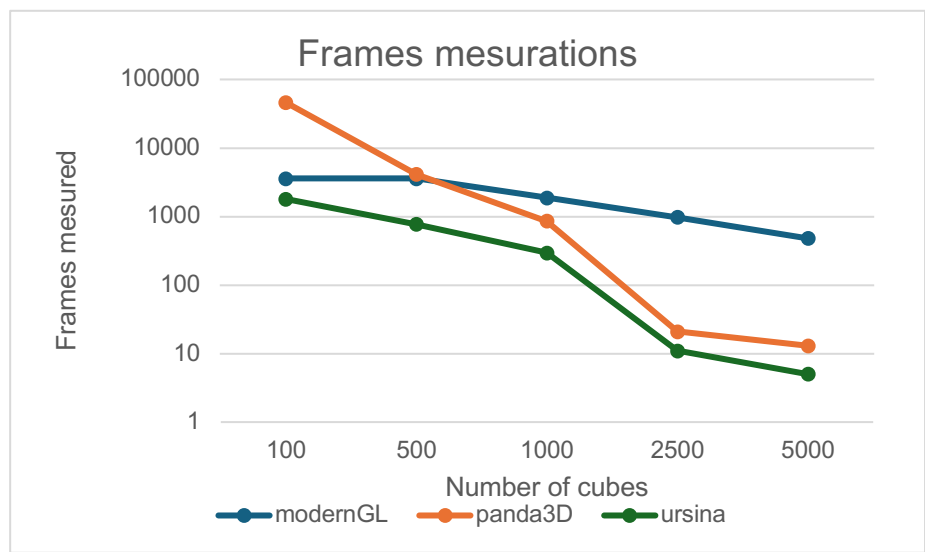
1000 cubes, the fluidity was the main difference. Only modernGL was still moving with as much fluidity as before. With 2500 cubes, I was able to do a real ordering of observed speed from fast to too slow:

1. modernGL
2. panda3D
3. ursina

With 5000 cubes, I then realised how different the results were. ModernGL was still moving like with 100 cubes. It looked impressive. Panda3D was moving but slowly. Ursina was not really moving. Of course, it is unlikely that we will use this much "cubes" in our project, but it is still interesting to compare the libraries' adaptability.

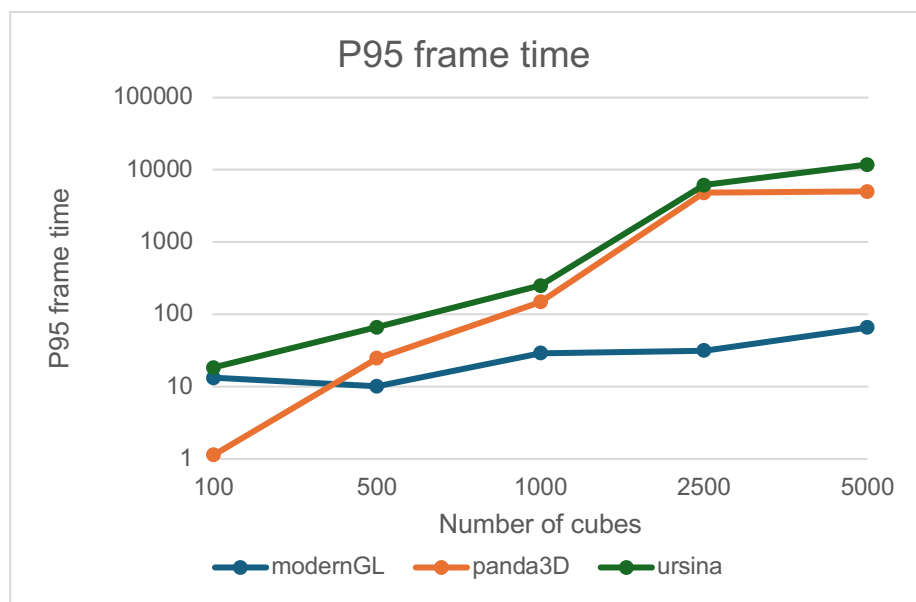
4. ANALYSIS OF RESULTS

Here are some graphs expressing the objective results from the experiments.



Here we can observe that ursina is always behind modernGL and panda3D. With few cubes panda3D is the best but with more cubes modernGL is the best with a higher difference. These values represent the time it takes for the cubes to move. Highest frames mensuration means faster library.

Note: The unusually constant frame rates observed for Ursina (~60 FPS) and ModernGL (~120 FPS) suggest that frame synchronization may have affected the measurements. Therefore, these results should currently be interpreted as preliminary rather than as a definitive raw-performance comparison.



ModernGL maintains comparatively low P95 frame times as scene complexity increases. Panda3D performs very well with a small number of cubes, but its P95 frame time increases considerably for more complex scenes. Ursina shows a similar degradation.

5. REVIEW

Here is a small review of all the good and bad points of each library.

A. MODERNGL

ModernGL showed the best performance under high scene complexity in our benchmark. It obviously looks better than the others, but it looks hard to use. The code is a lot longer and harder to understand. The main point here is: Will we be able to use it? Isn't it too ambitious?

B. PANDA3D

Panda3D is the fastest with only few cubes, but do we need more cubes? 100 is already big. In addition, this library is the only one for which I had no error while trying to run it. But it is still the less stable one.

C. URSINA

Ursina is maybe the easier one. The code is concise and looks easy. Even more, the stability is not this bad, but its speed is! It has the worst speed, and I had some issues while trying to run it.

6. CONCLUSION

In this last part of my analysis, I will only give my opinion. I think ursina is the worst choice. Even if it looks easy, it is too slow and I had too many errors. I would not recommend modernGL as my first choice because, even if it looks perfect, I am pretty sure we won't be able to use it easily enough. Panda3D is my favourite. It is a nice balance. It does not look this hard to use and it is still fast. The instability is maybe the only thing we should pay attention to. So, here is my top 3:

1. Panda3D
2. ModernGL
3. Ursina

We still must talk about it. Tell me what you think of it!